

Week 3 AI Learning Plan

RAG Pipeline — 5 days Mon-Fri — ~2 hrs/day — LangChain + ChromaDB + Streamlit

NLP layer

LLM layer

GenAI layer

YouTube

GFG

Free Course

Week 3 goal

A working Streamlit app where you upload any PDF and chat with it using hybrid search + reranking. Upload your org documents — it answers questions with source page numbers. This is what you show your manager.

WEEK 3 OVERVIEW

■ click any row to jump to that day

Day 1	Embeddings + vector store ChromaDB setup and similarity search	~2 hrs
Day 2	Full RAG chain create_retrieval_chain + grounded answers	~2 hrs
Day 3	Document Q&A on real org documents Your org project starts here	~2 hrs
Day 4	Hybrid search + reranking Fix the weak answers from Day 3	~2 hrs
Day 5	Conversational RAG + Streamlit UI Your Week 3 deliverable	~2 hrs

Install everything once before Day 1

```
pip install langchain langchain-google-genai langchain-community
pip install chromadb pypdf flashrank rank_bm25 streamlit
```

Day 1

Embeddings + vector store | ChromaDB setup and similarity search

~2 hrs

ORG CONNECTION:

ChromaDB stores your org documents as vectors — this is the search engine behind every org RAG chatbot

NLP layer

Theory — How embeddings power RAG search

TOPICS IN THIS SECTION:

Embeddings

ChromaDB

similarity_search()

Vector store

Cosine similarity

Semantic search

TOPICS TO COMPLETE — check each as you understand it

☒ Why embeddings capture meaning — not just keywords

☒ How cosine similarity finds closest vectors to a query

☒ What ChromaDB stores — vectors + metadata + original text

☒ Why vector search is faster than comparing all chunks

☒ How persist_directory saves your vector store to disk

Embeddings convert text into vectors that capture meaning. Similar meaning = close vectors in space. A vector store like ChromaDB stores these vectors and finds the closest ones to your query in milliseconds. This retrieval step is what makes RAG work — it finds the right document chunks before passing them to the LLM.

FOCUS ON

Embedding = text as numbers | Closer vectors = more similar meaning | ChromaDB = the search engine for your documents

SKIP FOR NOW

FAISS, Pinecone, Weaviate for now — ChromaDB is free and local

RESOURCES

☐

YouTube

Krish Naik — Complete RAG Crash Course with LangChain (2 hrs)

☐

YouTube

Master RAG with LangChain and ChromaDB — Python Tutorial

☐

Blog

Vector Stores in LangChain — GFG

☐

Course

DeepLearning.AI — LangChain Chat with Your Data (free)

READ MORE — TOPIC REFERENCES

GeeksForGeeks

Vector Stores in LanoChain — ChromaDB. FAISS. similairtv search

GeeksForGeeks

RAG with LangChain — full pipeline overview

GeeksForGeeks

Retrievals in LangChain — retriever setup and query

GeeksForGeeks

Word Embeddings in NLP — how vectors represent meaning

— Take a 10 min break before starting practice —

GenAI layer

Practice — Embed a PDF into ChromaDB and run similarity search

TOPICS IN THIS SECTION:

GoogleGenerativeAIEmbeddings

Chroma.from_documents()

similarity_search()

persist_directory

PyPDFLoader

TOPICS TO COMPLETE — check each as you finish the task

☒

 pip install langchain langchain-google-genai langchain-community chromadb pypdf

☒

 Load a PDF with PyPDFLoader and check page count

☒

 Split into chunks with RecursiveCharacterTextSplitter (chunk_size=500)

☒

 Create GoogleGenerativeAIEmbeddings with gemini-embedding-001

☒

 Store chunks in ChromaDB with Chroma.from_documents()

☒

 Run similarity_search() with 3 different queries

☒

 Inspect returned chunks — check page number in metadata

WATCH BEFORE CODING

YouTube

CONCEPTS

Master RAG with LangChain and ChromaDB (Jan 2025)

Free Course

OPENAI

DeepLearning.AI — LangChain Chat with Your Data (free)

READ MORE — TOPIC REFERENCES (verified links)

GFG

GFG — Vector Stores in LangChain

GFG

GFG — Retrievals in LangChain

COMMON ERRORS + QUICK FIXES

ModuleNotFoundError: chromadb

 → Run: pip install chromadb --upgrade

ValueError: embed_content dimension mismatch

 → Delete chroma_week3 folder and re-embed — dimension changed

Empty page_content in results

 → PDF might be scanned/image-based — needs OCR, use a text-based

Quota exceeded on embeddings

 → Free tier has limits — add delay: import time; time.sleep(1) b

Step 1

Install and load a PDF

New notebook. Install packages, load any PDF, check what you get.

```

pip install langchain langchain-google-genai langchain-community chromadb pypdf

from langchain_community.document_loaders import PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter

loader = PyPDFLoader("your_document.pdf") # any PDF
pages = loader.load()
print(f"Loaded {len(pages)} pages")
print(f"First 300 chars: {pages[0].page_content[:300]}")
print(f"Metadata: {pages[0].metadata}")

```

Note: Week 2 connection: you built PyPDFLoader on Day 4. Today you take it further — embedding and searching those pages.

Step 2 Chunk and embed into ChromaDB

New cell. This creates your searchable document database.

```

from langchain_google_genai import GoogleGenerativeAIEmbeddings
from langchain_community.vectorstores import Chroma

splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=50)
chunks = splitter.split_documents(pages)
print(f"Created {len(chunks)} chunks")

embeddings = GoogleGenerativeAIEmbeddings(
    model="models/gemini-embedding-001",
    google_api_key="your-key-here"
)

vectorstore = Chroma.from_documents(
    documents=chunks,
    embedding=embeddings,
    persist_directory="./chroma_week3"
)

print(f"Embedded and stored in ChromaDB")

```

Week 2 connection: `persist_directory` saves to disk so you can reload without re-embedding every time. Save your API costs.

Step 3 Run similarity search — see RAG retrieval in action

New cell. Ask questions and see which chunks come back.

```

queries = [
    "What is the main topic of this document?",
    "What are the key recommendations?",
    "What problems does it describe?"
]

for query in queries:
    print(f"Query: {query}")
    results = vectorstore.similarity_search(query, k=3)
    for i, doc in enumerate(results):
        page = doc.metadata.get("page", "?")
        print(f" [{i+1}] Page {page}: {doc.page_content[:150]}...")
    print()

```

Note: These retrieved chunks are exactly what gets passed to the LLM in Day 2. The quality of retrieval determines the quality of the final answer.

End of Day 1 — mark each one when done

- ☒ PDF loaded and page count checked
- ☒ Chunks created and average size printed
- ☒ ChromaDB created with persist_directory
- ☒ similarity_search() returns relevant chunks for 3 different queries
- ☒ I checked the metadata to see page numbers on returned chunks
- ☒ Notebook saved as week3_day1_vectorstore.ipynb

NEXT: Day 2: Connect the vector store to the LLM — build the full RAG chain with create_retrieval_chain. Ask a question and get a grounded answer with sources.

MY NOTES — write your key takeaway before moving on

Takeaway: _____

Surprised by: _____

Question: _____

Day 2 Full RAG chain | create_retrieval_chain + grounded answers

~2 hrs

ORG CONNECTION: create_retrieval_chain is the core of every org document Q&A app — connect it to your policy docs next

LLM layer

Theory — RAG architecture — retrieval → context → generation

TOPICS IN THIS SECTION:

RAG architecture

create_stuff_documents_chain

Context injection

create_retrieval_chain

Retriever

Grounded answers

TOPICS TO COMPLETE — check each as you understand it

- ☒ RAG flow: query → retriever → chunks → prompt → LLM → answer
- ☒ What create_stuff_documents_chain does — stuffs chunks into prompt
- ☒ What create_retrieval_chain does — links retriever + document chain
- ☒ How result["answer"] and result["context"] differ
- ☒ Why RAG answers are more reliable than bare LLM answers
- ☒ What happens when no relevant chunk is found

A RAG chain connects three things: a retriever that finds relevant chunks, a prompt template that injects those chunks as context, and an LLM that generates an answer using only that context. LangChain's create_retrieval_chain handles the wiring. The result["context"] tells you which chunks were used. The result["answer"] is the grounded response.

FOCUS ON Retriever = search, document chain = format + generate | result["context"] shows you which chunks the LLM used

SKIP FOR NOW Map-reduce chain, refine chain — use stuff chain for now

RESOURCES

☐

YouTube

Complete RAG System Tutorial LangChain + ChromaDB (Jun 2025)

☐

YouTube

RAG Tutorial with LangChain — Complete Tutorial (Feb 2025)

☐

Blog

RAG with LangChain — GFG full guide

☐

Blog

Retrievals in LangChain — GFG

READ MORE — TOPIC REFERENCES

GeeksForGeeks

RAG with LangChain — step by step pipeline

GeeksForGeeks

Retrievals in LangChain — create_retrieval_chain

GeeksForGeeks

Introduction to LangChain — refresher

GeeksForGeeks

LangChain APIs and Environment Setup

— Take a 10 min break before starting practice —

GenAI layer

Practice — Build the full RAG chain and get grounded answers

TOPICS IN THIS SECTION:

create_retrieval_chain

as_retriever()

result["context"]

create_stuff_documents_chain

result["answer"]

TOPICS TO COMPLETE — check each as you finish the task

☒

 Load vectorstore from persist_directory (reuse Day 1)

☒

 Create retriever with vectorstore.as_retriever(search_kwargs={"k":4})

☒

 Write system prompt with {context} and {input} placeholders

☒

 Build create_stuff_documents_chain(llm, prompt)

☒

 Build create_retrieval_chain(retriever, document_chain)

☒

 Ask 5 questions — print answer and source page numbers

☒

 Ask a question not in the document — verify "not found" response

WATCH BEFORE CODING

YouTube

CONCEPTS

Complete RAG System Tutorial — LangChain + ChromaDB (Jun 2025)

Free Course

OPENAI

DeepLearning.AI — LangChain Chat with Data (retrieval lessons)

READ MORE — TOPIC REFERENCES (verified links)

GFG

GFG — RAG with LangChain

GFG

GFG — Retrievals in LangChain

COMMON ERRORS + QUICK FIXES

InvalidArgument: Request contains an invalid argumen

 → Check model name is "gemini-2.0-flash" not "gemini-pro"

result["answer"] is empty

 → Retriever returned no chunks — check vectorstore path and re-e

context placeholder error in prompt

 → System prompt must have {context}, human must have {input} — e

Chroma collection not found

 → persist_directory path must match exactly what you used in Day

Step 1 Build the full RAG chain

New notebook. Load your Day 1 vector store and connect it to the LLM.

```

from langchain_google_genai import ChatGoogleGenerativeAI, GoogleGenerativeAIEmbeddings
from langchain_community.vectorstores import Chroma
from langchain.chains import create_retrieval_chain
from langchain.chains.combine_documents import create_stuff_documents_chain
from langchain_core.prompts import ChatPromptTemplate

llm = ChatGoogleGenerativeAI(model="gemini-2.0-flash", google_api_key="your-key")
embeddings = GoogleGenerativeAIEmbeddings(
    model="models/gemini-embedding-001", google_api_key="your-key")

# Load existing vectorstore from Day 1
vectorstore = Chroma(
    persist_directory="./chroma_week3",
    embedding_function=embeddings
)
retriever = vectorstore.as_retriever(search_kwargs={"k": 4})

system_prompt = """Answer ONLY using the provided context.
If the answer is not in the context, say "I cannot find this in the document."

Context:
{context}"""

prompt = ChatPromptTemplate.from_messages([
    ("system", system_prompt),
    ("human", "{input}")
])

doc_chain = create_stuff_documents_chain(llm, prompt)
rag_chain = create_retrieval_chain(retriever, doc_chain)
print("RAG chain ready")

```

Week 2 connection: Week 2 connection: you used `ConversationChain` for memory and chains for prompts. RAG chain is the same pattern — prompt + LLM — but with a retriever plugged in to supply the context automatically.

Step 2 Ask questions and check sources

New cell. This is the first time you get a real grounded answer with citations.

```

questions = [
    "What is the main topic of this document?",
    "What are the key recommendations?",
    "What are the limitations mentioned?",
    "What year was this published?", # might not be in doc
    "What is the weather like today?", # definitely not in doc
]

for q in questions:
    result = rag_chain.invoke({"input": q})
    pages = sorted(set(d.metadata.get("page", "?") for d in result["context"]))
    print(f"Q: {q}")
    print(f"A: {result['answer'][:200]}")
    print(f"Sources: pages {pages}\n")

```

Note: The last two questions should return "I cannot find this in the document." This is the guardrail you get for free from the system prompt — a huge improvement over a bare LLM that would hallucinate an answer.

End of Day 2 — mark each one when done

- ☒ RAG chain built with `create_retrieval_chain` — runs without errors
- ☒ Asking 5 questions returns grounded answers with source page numbers
- ☒ A question NOT in the document returns the fallback message
- ☒ I understand what `result["answer"]` and `result["context"]` contain
- ☒ Notebook saved as `week3_day2_rag_chain.ipynb`

NEXT: Day 3: Load your real org document. Ask 10 actual questions about it. Check answer quality and fix the prompt if needed. This is your org project.

MY NOTES — write your key takeaway before moving on

Takeaway: _____
Surprised by: _____
Question: _____

Day 3 Document Q&A on real org documents | Your org project starts here

~2 hrs

ORG CONNECTION: This is your org project — a working Q&A system on real company documents, ready to show your manager

LLM layer **Theory — Prompt design for RAG + source attribution**

TOPICS IN THIS SECTION:

- | | |
|---|--|
| ☒ RAG prompt design | ☒ Source attribution |
| ☒ Hallucination | ☒ Grounding |
| ☒ Fallback response | ☒ Citation format |

TOPICS TO COMPLETE — check each as you understand it

- ☒ Why RAG prompts need explicit "use only the context" instruction
- ☒ How to write a fallback for missing information
- ☒ How to instruct the model to cite page numbers in its answer
- ☒ Why chunk quality affects answer quality more than prompt quality
- ☒ How to test RAG quality — questions with known answers

The system prompt in RAG is more important than in regular chatbots. It controls: whether the LLM stays in-document, how it formats answers, and how it cites sources. Good RAG prompts include three things: a strict instruction to use only the context, a fallback instruction for missing information, and a citation format so users know where answers came from.

FOCUS ON System prompt quality directly determines answer quality | Always test with questions whose answers you know first

SKIP FOR NOW RAGAS evaluation framework — add this in Week 4 if needed

RESOURCES

- ☐ **YouTube** Krish Naik — Complete RAG Crash Course with LangChain 2025
- ☐ **Blog** RAG with LangChain — GFG complete guide
- ☐ **Blog** RAGAS — RAG evaluation framework GFG

READ MORE — TOPIC REFERENCES

GeeksForGeeks

RAG with LangChain — prompt design section

GeeksForGeeks

Retrievals in LangChain

GeeksForGeeks

RAGAS — evaluate your RAG quality

— Take a 10 min break before starting practice —

GenAI layer

Practice — Load your org document and test with real questions

TOPICS IN THIS SECTION:

Org document

Page citations

Answer quality

Real questions

RAG pipeline

TOPICS TO COMPLETE — check each as you finish the task

☒

 Choose a real org document — policy manual, FAQ, HR guide, product doc

☒

 Load, chunk, embed into a new ChromaDB collection

☒

 Write an improved system prompt with citation instruction

☒

 Ask 10 real questions — ones employees would actually ask

☒

 Print answer + source pages for each question

☒

 Identify where answers are wrong or incomplete

☒

 Adjust chunk_size or k to improve weak answers

WATCH BEFORE CODING

YouTube

CONCEPTS

Krish Naik — RAG Crash Course with LangChain 2025 (2 hrs)

YouTube

CONCEPTS

Production RAG with LangChain and Vector DBs — Full Course

READ MORE — TOPIC REFERENCES (verified links)

GFG

GFG — RAG with LangChain

GFG

GFG — RAGAS — evaluation

COMMON ERRORS + QUICK FIXES

PDF is scanned and gives garbage text

→ Use a text-based PDF — scanned PDFs need OCR (pytesseract) whi

Answers are too vague or generic

→ Reduce chunk_size to 300 and increase k to 6 — get more specif

Page numbers show as ? in sources

→ PyPDFLoader adds page numbers automatically — check pages[0].m

LLM ignores the citation instruction

→ Add "Format: (Source: page X)" at the end of your system promp

Step 1 Load your real org document

Replace the sample PDF with something from your actual work.


```

from langchain_community.document_loaders import PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_google_genai import GoogleGenerativeAIEmbeddings
from langchain_community.vectorstores import Chroma

# Load YOUR org document
loader = PyPDFLoader("your_org_document.pdf") # HR policy, manual, FAQ...
pages = loader.load()
print(f"Org doc: {len(pages)} pages")

chunks = RecursiveCharacterTextSplitter(
    chunk_size=500, chunk_overlap=50
).split_documents(pages)

embeddings = GoogleGenerativeAIEmbeddings(
    model="models/gemini-embedding-001", google_api_key="your-key")
vectorstore = Chroma.from_documents(
    chunks, embeddings, persist_directory="./chroma_org_doc")
print(f"Ready: {len(chunks)} chunks embedded")

```

Week 2 connection: If you don't have an org document yet, use any professional PDF — an annual report, product manual, or technical specification works perfectly.

Step 2 Build RAG with citation-aware prompt

New cell. This prompt tells the model to include page numbers in its answer.

```

from langchain_google_genai import ChatGoogleGenerativeAI
from langchain.chains import create_retrieval_chain
from langchain.chains.combine_documents import create_stuff_documents_chain
from langchain_core.prompts import ChatPromptTemplate

llm = ChatGoogleGenerativeAI(model="gemini-2.0-flash", google_api_key="your-key")
retriever = vectorstore.as_retriever(search_kwargs={"k": 4})

system_prompt = """You are a helpful assistant answering questions about company documents
Rules:
1. Answer ONLY from the context provided below
2. Always mention which page number(s) your answer comes from
3. If not in context, say: "This information is not in the document."
4. Be concise — 2-3 sentences maximum

Context:
{context}"""

prompt = ChatPromptTemplate.from_messages([
    ("system", system_prompt), ("human", "{input}")
])
rag_chain = create_retrieval_chain(
    retriever, create_stuff_documents_chain(llm, prompt))

```

Step 3 Ask 10 real questions — evaluate quality

New cell. These should be questions real users would ask about your document.

```
# Replace with questions relevant to YOUR document
questions = [
    "What is the leave policy for sick days?",
    "How many days of annual leave do employees get?",
    "What is the process for claiming expenses?",
    "Who do I contact for IT support?",
    "What is the notice period for resignation?",
    "What is not mentioned in this document?", # test fallback
]

print("=" * 60)
for q in questions:
    result = rag_chain.invoke({"input": q})
    pages = sorted(set(d.metadata.get("page", "?") for d in result["context"]))
    print(f"Q: {q}")
    print(f"A: {result['answer']}")
    print(f"Sources: pages {pages}\n")
```

Note: Write down which questions gave wrong or incomplete answers. Bring that list to Day 4 — hybrid search and reranking fix exactly these cases.

End of Day 3 — mark each one when done

- ☒ Real org document loaded and embedded into ChromaDB
- ☒ Citation-aware system prompt built
- ☒ Asked 10 real questions — printed answers with page numbers
- ☒ At least one question returned the "not in document" fallback correctly
- ☒ I noted which questions gave weak answers for Day 4 tuning
- ☒ Notebook saved as week3_day3_org_rag.ipynb

NEXT: Day 4: Fix the weak answers from today. Hybrid search adds BM25 keyword matching. Reranking re-scores all retrieved chunks to find the truly best ones. These two changes fix 70% of RAG failures.

MY NOTES — write your key takeaway before moving on

Takeaway: _____

Surprised by: _____

Question: _____

Day 4 Hybrid search + reranking | Fix the weak answers from Day 3

~2 hrs

ORG CONNECTION: Hybrid search fixes org-specific codes and IDs that pure vector search misses — critical for enterprise documents

NLP layer

Theory — Hybrid search and reranking — the two biggest RAG improvements

TOPICS IN THIS SECTION:

Hybrid search

EnsembleRetriever

Cross-encoder

BM25

Reranking

MultiQueryRetriever

TOPICS TO COMPLETE — check each as you understand it

☒

 Why pure vector search fails on exact terms and codes

☒

 How BM25 keyword search complements vector search

☒

 EnsembleRetriever — how weights balance BM25 vs vector

☒

 How a cross-encoder reranker re-scores chunks (not cosine similarity)

☐

 FlashRank — free local reranker, no extra API key needed

☐

 MultiQueryRetriever — generates 3 query variants to get diverse results

Plain vector search misses exact terms like "HR-2024-07" or employee codes. BM25 keyword search finds them but misses semantic meaning. Hybrid search combines both using EnsembleRetriever. Reranking then takes the top-20 candidates and re-scores each against the query using a cross-encoder model — much more accurate than cosine similarity alone. Together these fix most RAG failures.

FOCUS ON

Hybrid search = BM25 + vector (best of both) | Reranking = second pass that finds the truly best chunks

SKIP FOR NOW

Cohere Rerank API, ColBERT, learned sparse retrieval — advanced, use FlashRank first

RESOURCES

☐

YouTube

Production RAG: Hybrid Search + Reranking + HyDE (Aug 2025)

☐

YouTube

Hybrid Search RAG in 10 Minutes — BM25 + Vector (Oct 2025)

☐

YouTube

RAG Reranking Explained — How to Improve RAG Results (Oct 2025)

☐

Blog

RAG with LangChain — advanced retrieval GFG

READ MORE — TOPIC REFERENCES

GeeksForGeeks

RAG with LangChain — hybrid and advanced retrieval

GeeksForGeeks

Retrievals in LangChain — EnsembleRetriever

GeeksForGeeks

BM25 — beyond TF-IDF for smarter retrieval

— Take a 10 min break before starting practice —

GenAI layer

Practice — Add hybrid search and reranking to your Day 3 RAG app

TOPICS IN THIS SECTION:

BM25Retriever

FlashrankRerank

MultiQueryRetriever

EnsembleRetriever

ContextualCompressionRetriever

TOPICS TO COMPLETE — check each as you finish the task

☒

 pip install flashrank rank_bm25

☒

 Create BM25Retriever from your document chunks

☒

 Create EnsembleRetriever (BM25 weight=0.4, vector weight=0.6)

☒

 Add FlashrankRerank on top of the ensemble retriever

☒

 Test with the same weak questions from Day 3

☒

 Compare plain vector vs hybrid+reranked results side by side

☒

 Optional: add MultiQueryRetriever for broader coverage

WATCH BEFORE CODING

YouTube

CONCEPTS

Production RAG: Hybrid Search + Reranking + HyDE (Aug 2025)

YouTube

CONCEPTS

Hybrid Search RAG in 10 Minutes — BM25 + Vector (Oct 2025)

READ MORE — TOPIC REFERENCES (verified links)

GFG

[GFG — RAG with LangChain \(hybrid section\)](#)

GFG

[GFG — Retrievals in LangChain](#)**COMMON ERRORS + QUICK FIXES****ImportError: rank_bm25 not found** → Run: `pip install rank_bm25` (separate from flashrank)**FlashrankRerank error on first run** → It downloads a small model on first use — needs internet, wait**EnsembleRetriever weights must sum to 1** → `weights=[0.4, 0.6]` — they must add up to exactly 1.0**Reranking returns fewer chunks than expected** → `top_n=3` in FlashrankRerank limits output — increase to 5 if ne**Step 1** Install hybrid search dependencies

New cell. Two extra packages for BM25 and reranking.

```

pip install flashrank rank_bm25

# Verify imports work
from langchain_community.retrievers import BM25Retriever
from langchain.retrievers import EnsembleRetriever
from langchain.retrievers import ContextualCompressionRetriever
from langchain.retrievers.document_compressors import FlashrankRerank
from langchain.retrievers.multi_query import MultiQueryRetriever
print("All imports successful")

```

Step 2 Build hybrid retriever (BM25 + vector)

New cell. Use your Day 3 chunks and vectorstore.

```

from langchain_community.retrievers import BM25Retriever
from langchain.retrievers import EnsembleRetriever

# BM25 retriever (keyword search)
bm25_retriever = BM25Retriever.from_documents(chunks)
bm25_retriever.k = 4

# Vector retriever (semantic search)
vector_retriever = vectorstore.as_retriever(search_kwargs={"k": 4})

# Hybrid: 40% keyword + 60% semantic
hybrid_retriever = EnsembleRetriever(
    retrievers=[bm25_retriever, vector_retriever],
    weights=[0.4, 0.6]
)
print("Hybrid retriever ready")

```

Note: The 0.4/0.6 weight split is a good starting point. Increase BM25 weight (0.6/0.4) if your documents have lots of exact codes or IDs.**Step 3** Add reranking on top

New cell. Reranker re-scores all retrieved chunks — picks the truly best ones.

```

from langchain.retrievers import ContextualCompressionRetriever
from langchain.retrievers.document_compressors import FlashrankRerank

# FlashRank reranker – free, runs locally, no API key needed
compressor = FlashrankRerank(top_n=3) # keep best 3 after reranking
reranking_retriever = ContextualCompressionRetriever(
    base_compressor=compressor,
    base_retriever=hybrid_retriever
)

# Compare: plain vector vs hybrid+reranked
weak_query = "your weak question from Day 3"
plain = vectorstore.similarity_search(weak_query, k=3)
better = reranking_retriever.get_relevant_documents(weak_query)

print("PLAIN VECTOR:")
for d in plain: print(f" {d.page_content[:120]}...")
print("\nHYBRID + RERANKED:")
for d in better: print(f" {d.page_content[:120]}...")

```

Week 2 connection: The reranked results should be noticeably more relevant to your query. If they look the same, try a more specific question that has exact keywords.

Step 4 Rebuild RAG chain with improved retriever

New cell. Plug the reranking retriever into your Day 2 RAG chain.

```

from langchain.chains import create_retrieval_chain
from langchain.chains.combine_documents import create_stuff_documents_chain

# Full chain with hybrid + reranking
improved_rag = create_retrieval_chain(
    reranking_retriever,
    create_stuff_documents_chain(llm, prompt) # same prompt from Day 3
)

# Test the same weak questions from Day 3
for q in weak_questions_from_day3:
    result = improved_rag.invoke({"input": q})
    print(f"Q: {q}")
    print(f"A: {result['answer'][:200]}\n")

```

Note: Replace `weak_questions_from_day3` with the actual questions that gave poor answers yesterday. Document the improvement — this is what you show your team.

End of Day 4 — mark each one when done

- ☒ BM25Retriever and EnsembleRetriever built without errors
- ☒ FlashrankRerank added — reranking retriever runs
- ☒ Compared plain vector vs hybrid+reranked on the same weak question
- ☒ Rebuilt RAG chain using the improved retriever
- ☒ Tested with Day 3 weak questions — answers improved
- ☒ Notebook saved as week3_day4_hybrid_reranking.ipynb

NEXT: Day 5: Wrap everything in a Streamlit app — PDF upload, chat interface, source citations, clear history button. This is your Week 3 deliverable — a real shareable app.

MY NOTES — write your key takeaway before moving on

Takeaway:

Surprised by:

Question:

Day 5

Conversational RAG + Streamlit UI | Your Week 3 deliverable

~2 hrs

ORG CONNECTION: This Streamlit app is your Week 3 deliverable — upload any org document and let colleagues chat with it

LLM layer

Theory — Streamlit session state and conversational RAG

TOPICS IN THIS SECTION:

Streamlit

st.chat_input()

Chat history

Session state

Conversational RAG

File uploader

TOPICS TO COMPLETE — check each as you understand it

☐ Why Streamlit rerenders on every interaction

☐ What st.session_state stores and why it persists between rerenders

☐ How st.chat_input and st.chat_message work together

☐ Why conversational RAG needs chat_history in the retriever query

☐ How create_history_aware_retriever reformulates queries with history

☐ What st.spinner, st.success, st.sidebar do for UX

Streamlit rerenders the entire Python script on every user interaction. Session state preserves data (chat history, vector store) between rerenders. Without it, every message would re-embed the whole document. Conversational RAG adds chat history to the retrieval context so follow-up questions work correctly — "what did you just say about that?" actually gets the right answer.

FOCUS ON Session state = the memory of your Streamlit app | Conversational RAG = RAG that remembers the conversation

SKIP FOR NOW Streamlit Cloud deployment, authentication, multi-user sessions — Week 4 topic

RESOURCES

☐

YouTube

 RAG Onboarding Chatbot — Streamlit + LangChain + ChromaDB (Sep 2025)

☐

YouTube

 Streamlit RAG Chatbot App with PDF File Upload (Nov 2024)

☐

Blog

 Streamlit API reference — official docs

☐

Blog

 RAG with LangChain — GFG conversational section

READ MORE — TOPIC REFERENCES

GeeksForGeeks

RAG with LangChain — conversational RAG section

GeeksForGeeks

LangChain Memory — for chat history

GeeksForGeeks

Introduction to Streamlit — GFG beginner guide

— Take a 10 min break before starting practice —

GenAI layer

Practice — Build and run the Streamlit RAG app

TOPICS IN THIS SECTION:

st.chat_input()

st.session_state

st.file_uploader()

st.spinner()

Streamlit app

TOPICS TO COMPLETE — check each as you finish the task

☐ pip install streamlit

☐ Create streamlit_rag_app.py (separate file, not Jupyter)

☐ Add sidebar with API key input and PDF file uploader

☐ Add st.session_state for chat history and vectorstore

☐ Add st.chat_input for question input

☐ Display source pages with each answer using st.expander

☐ Run with: streamlit run streamlit_rag_app.py

WATCH BEFORE CODING

YouTube

CONCEPTS

RAG Onboarding Chatbot Streamlit+LangChain+ChromaDB (Sep 2025)

YouTube

CONCEPTS

Streamlit RAG Chatbot App with PDF Upload (Nov 2024)

READ MORE — TOPIC REFERENCES (verified links)

GFG

Streamlit official docs — API reference

GFG

GFG — RAG with LangChain

COMMON ERRORS + QUICK FIXES

streamlit not found

→ Run: pip install streamlit — then restart terminal

App re-embeds on every message

→ Check the session_state condition — "vectorstore" not in st.se

File uploader returns None after upload

→ Normal — Streamlit rerenders, use session_state.doc_name to tr

App crashes on large PDF (50+ pages)

→ Chunk and embed in batches — split into groups of 20 pages bef

Step 1 Create the Streamlit app file

New FILE (not Jupyter) — save as streamlit_rag_app.py. Run with: streamlit run streamlit_rag_app.py

```

import streamlit as st
from langchain_google_genai import ChatGoogleGenerativeAI, GoogleGenerativeAIEmbeddings
from langchain_community.vectorstores import Chroma
from langchain_community.document_loaders import PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain.chains import create_retrieval_chain
from langchain.chains.combine_documents import create_stuff_documents_chain
from langchain_core.prompts import ChatPromptTemplate
import tempfile, os

st.set_page_config(page_title="Document Q&A", layout="centered")
st.title("Ask Your Documents")

with st.sidebar:
    api_key = st.text_input("Gemini API Key", type="password")
    uploaded = st.file_uploader("Upload PDF", type="pdf")
    if st.button("Clear Chat"):
        st.session_state.messages = []
        st.rerun()

```

Step 2 Add document processing and RAG chain setup

Continue in the same file — add after the sidebar block.

```

if uploaded and api_key:
    if "vectorstore" not in st.session_state or \
        st.session_state.get("doc_name") != uploaded.name:
        with tempfile.NamedTemporaryFile(delete=False, suffix=".pdf") as f:
            f.write(uploaded.read()); tmp = f.name
        with st.spinner(f"Processing {uploaded.name}..."):
            chunks = RecursiveCharacterTextSplitter(
                chunk_size=500, chunk_overlap=50
            ).split_documents(PyPDFLoader(tmp).load())
            embeddings = GoogleGenerativeAIEmbeddings(
                model="models/gemini-embedding-001", google_api_key=api_key
            )
            st.session_state.vectorstore = Chroma.from_documents(chunks, embeddings)
            st.session_state.doc_name = uploaded.name
            st.session_state.chunks = chunks
        os.unlink(tmp)
        st.success(f"Ready: {len(chunks)} chunks from {uploaded.name}")

    llm = ChatGoogleGenerativeAI(model="gemini-2.0-flash", google_api_key=api_key)
    retriever = st.session_state.vectorstore.as_retriever(search_kwargs={"k":4})
    prompt = ChatPromptTemplate.from_messages([
        ("system", "Answer ONLY from context. Cite page numbers.\n{context}"),
        ("human", "{input}")
    ])
    rag_chain = create_retrieval_chain(
        retriever, create_stuff_documents_chain(llm, prompt))

```

Week 2 connection: The `session_state` check prevents re-embedding on every message — crucial for performance and API cost saving.

Step 3 Add chat interface

Continue in the same file — add at the end.


```

if "messages" not in st.session_state:
    st.session_state.messages = []

for msg in st.session_state.messages:
    st.chat_message(msg["role"]).write(msg["content"])

if q := st.chat_input("Ask about your document..."):
    st.session_state.messages.append({"role": "user", "content": q})
    st.chat_message("user").write(q)

    with st.spinner("Thinking..."):
        result = rag_chain.invoke({"input": q})

    pages = sorted(set(d.metadata.get("page", "?") for d in result["context"]))
    ans = f"{result['answer']}\n\n*Sources: pages {pages}*"

    st.session_state.messages.append({"role": "assistant", "content": ans})
    st.chat_message("assistant").write(ans)

else:
    st.info("Add your Gemini API key and upload a PDF to start.")

```

Note: Run with: `streamlit run streamlit_rag_app.py` — your browser opens automatically. Upload your org document and start chatting with it.

End of Day 5 — mark each one when done

- ☐ `streamlit_rag_app.py` created and runs with `streamlit run` command
- ☐ PDF upload works — app processes and embeds the document
- ☐ Chat interface shows questions and answers correctly
- ☐ Source page numbers appear with every answer
- ☐ Uploading a second document replaces the first without crashing
- ☐ Clear chat button works — resets conversation history
- ☐ App tested with the org document from Day 3

NEXT: Week 4: RAG quality tuning, LangSmith debugging, error handling and guardrails, Streamlit polish, and team demo. Your app goes from working to production-ready.

MY NOTES — write your key takeaway before moving on

Takeaway: _____

Surprised by: _____

Question: _____

Week 3 Resources — All Verified Links

Every link below has been verified working. No GitHub links. Click directly from this page.

YouTube — Week 3 specific (all verified 2024-2026)

- ☐ [Krish Naik — Complete RAG Crash Course with LangChain \(Oct 2025, 2 hrs\)](#)
- ☐ [Master RAG with LangChain and ChromaDB — Python Tutorial \(Jan 2025\)](#)
- ☐ [Complete RAG System Tutorial — LangChain + ChromaDB \(Jun 2025\)](#)
- ☐ [Production RAG: Hybrid Search + Reranking + HyDE \(Aug 2025\)](#)
- ☐ [Hybrid Search RAG in 10 Minutes — BM25 + Vector \(Oct 2025\)](#)
- ☐ [RAG Reranking Explained: How to Improve RAG Results \(Oct 2025\)](#)
- ☐ [RAG Onboarding Chatbot Streamlit + LangChain + ChromaDB \(Sep 2025\)](#)
- ☐ [Streamlit RAG Chatbot App with PDF File Upload \(Nov 2024\)](#)

GFG — reading references (all verified)

- ☐ [RAG with LangChain — full pipeline guide](#)
- ☐ [Vector Stores in LangChain — ChromaDB and FAISS](#)
- ☐ [Retrievals in LangChain — create_retrieval_chain](#)
- ☐ [RAGAS — evaluate your RAG quality](#)
- ☐ [BM25 — beyond TF-IDF for smarter retrieval](#)
- ☐ [Introduction to Streamlit](#)

Free courses and official docs

- ☐ [DeepLearning.AI — LangChain Chat with Your Data \(free\)](#)
- ☐ [DeepLearning.AI — LangChain for LLM App Dev \(free\)](#)
- ☐ [Streamlit official API reference](#)
- ☐ [Streamlit session_state docs](#)